MODULE 27 · DOMAIN 3 - DEPLOYMENT AND ORCHESTRATION

Agentic workflows

The agent loop, Agents Classic, and the AgentCore components that replaced it.

MODULE 27 of 43 DOMAIN 3 - Deploy WEIGHT 24%

admin@sthithapragna.com

A model that can **act**, observe what happened, and decide what to do next.

What an agent does on every step

01
Observe
The goal, the history, and any results so far.

›

02
Plan
Decide the next action, or that the task is finished.

›

03
Act
Call a tool, query a knowledge base, or answer.

›

04
Evaluate
Read the result. Was it what was needed?

›

05
Repeat
Until the goal is met, or a limit is reached.

ON THE EXAM

Every arrow back to the top is another model call. That is why module 25 said capacity scales with steps, not users.

Bedrock Agents is now Agents Classic

- **Agents Classic is in maintenance mode.** AWS publishes a page documenting exactly that.
- It remains named in the exam guide as Amazon Bedrock Agents, so it remains **examinable**.
- AWS also publishes a **feature-by-feature migration mapping** to its successor, AgentCore.
- Learn the Classic concepts for the exam, and the AgentCore components for the job.
- The same pattern as Ground Truth, Clarify, Debugger and Model Monitor in earlier modules.

> ON THE EXAM
>
> This is the fifth maintenance flag in the course. The pattern is now familiar: examinable, superseded, and the successor is documented.

The concepts the exam guide names

CONCEPT	WHAT IT DOES
Agent	The orchestrating model, with instructions describing its purpose
Action group	A set of operations the agent may call, with an API schema
Lambda function	Executes the action group's operations against your systems
Knowledge base association	The agent can retrieve from a knowledge base as one of its actions
Session and memory	Idle TTL, memory types, and cross-session memory
Alias and version	A published version, and an alias that points at one

ON THE EXAM

Action groups are the key term. They are how a Classic agent is given the ability to do anything at all.

The pattern ladder, one last time

- **Prompt** when the model already knows enough. Module 16.
- **Retrieve** when the answer is in your documents. Module 26.
- **Tool use** when the answer is in a live system. Module 16.
- **Workflow** when the steps are known and fixed. Step Functions.
- **Agent** when the steps must be decided at run time. Only then.

ON THE EXAM

Five rungs, increasing in cost, unpredictability and operational burden. The exam consistently rewards the lowest one that works.

AgentCore is **modular**. Each component solves one problem agents create.

What AgentCore provides

COMPONENT	THE PROBLEM IT SOLVES
Runtime	Where the agent executes, and for how long
Gateway	Which tools and APIs the agent may reach, and under what permissions
Memory	What the agent remembers, within a session and across sessions
Identity	Who the agent is acting as, using OAuth 2.0 and OIDC
Observability	What the agent actually did, step by step
Policy	What the agent is allowed to do. GA March 2026
Evaluations	Whether it is any good. GA March 2026
Browser and Code Interpreter	Built-in tools: browse the web, run code in a sandbox

ON THE EXAM

Read this table as a list of the problems agents create. That is why the platform exists, and why agents are harder than they look.

AgentCore Policy

- Declarative rules about what an agent is **permitted** to do, enforced at run time.
- **Enforced on the Gateway**, so it applies to the tools rather than relying on the model's judgement.
- Generally available since **March 2026**.
- An instruction in a prompt is a request. A policy is a control.
- This is the agent equivalent of module 08's point about IAM versus a WHERE clause.

> ON THE EXAM
>
> Policy is where 'the agent must never delete anything' stops being a sentence in a prompt and starts being enforced.

AgentCore Evaluations

- Evaluation built for agents rather than for single responses. GA **March 2026**.
- Agents need **trajectory evaluation**: were the steps sensible, not just was the answer right?
- Module 18's list applies: wrong tool, bad arguments, loops, giving up early.
- Run it on a schedule, so a prompt or model change is caught before users find it.
- Two correct runs can differ legitimately, so a strict output match is the wrong test.

ON THE EXAM

For a model you evaluate one output. For an agent you evaluate a path, and several different paths can all be correct.

AgentCore Runtime

- Serverless, **microVM-based sessions**, with per-session isolation.
- The original ceiling was **eight hours** per session, which suits most interactive agents.
- **Runtime instances** add managed EC2 in your account, with sessions up to **fourteen days**.
- Those support **shared file systems, GPU instance types**, Python and container images.
- Multiple agents can share one runtime and collaborate through a **shared session directory**.

> ON THE EXAM
>
> The fourteen-day option exists because some agent work is genuinely long-running: migrations, research, multi-day analysis.

AgentCore Gateway

- A **tool registry** and access management layer for the APIs an agent may call.
- It is the successor to **action groups**, and it requires **explicit tool definition** rather than automatic elicitation.
- It carries a **permissions model**, so what an agent can reach is configuration rather than code.
- **Policy enforcement happens here**, which is where Guardrails and agent policy meet the tools.
- Module 16's warning applies: what you expose to an agent is a security decision.

> ON THE EXAM
>
> Every tool you register is a capability. The question from module 16 still holds: what is the worst thing if the agent is persuaded to call it?

Writing tools an agent can use well

- **The tool description is part of the prompt.** The model reads it to decide whether to call the tool.
- So a vague description produces a confidently wrong tool choice, and it looks like a model problem.
- Describe **when to use it** and **when not to**, not just what it does.
- Keep the input schema tight. Fewer optional parameters means fewer ways to call it wrongly.
- Return **useful errors**. The agent reads them and can correct itself if they say something.

> ON THE EXAM
>
> Tool descriptions are the most underrated prompt engineering in an agent. They decide every routing decision the agent makes.

Writing the agent's instructions

- State the **goal**, the **boundaries**, and what to do when it cannot proceed.
- **Define what finished means.** An agent with no completion condition will keep going.
- Say when to escalate to a human rather than guessing. That is a design decision, not a fallback.
- Instructions are guidance the model may deviate from. Policy and IAM are not.
- Module 16's prompt engineering applies, with one extra requirement: termination.

> ON THE EXAM
>
> The instruction most often missing is the one that says when to stop. Without it, 'keep trying' is a reasonable reading.

AgentCore Memory

- **Short-term memory** holds the current session: what has happened in this conversation.
- **Long-term memory** persists across sessions, so the agent remembers a user between conversations.
- It supports different **memory strategies**, including semantic, episodic and procedural.
- This replaces Classic's session configuration: idle TTL, memory types, cross-session memory.
- Persisted memory is **personal data**. Module 08 applies, including erasure obligations.

> ON THE EXAM
>
> Long-term memory is the feature users notice most, and the one with the heaviest compliance consequences.

AgentCore Identity

- Handles **OAuth 2.0 and OpenID Connect** flows so an agent can authenticate to systems.
- **Identity delegation:** the agent acts on behalf of a specific user, with that user's permissions.
- The alternative is an agent with one powerful service role, which can do anything for anyone.
- So a user who cannot see a record should not be able to make the agent fetch it.
- This is the agent equivalent of module 26's tenant filtering, applied to actions rather than documents.

> ON THE EXAM
>
> Without delegation, an agent's permissions are the union of everything any user might need. That is a large and permanent problem.

AgentCore Observability

- Built-in metrics for runtime, memory, gateway, built-in tools and identity, in **CloudWatch**.
- To see spans and traces you must first enable **CloudWatch Transaction Search**. One-time setup.
- For the full range, and for custom runtime metrics, instrument with the **ADOT SDK**.
- **Tracing must be enabled when a memory is created** to get service-provided spans for it.
- The point: reconstructing **why** an agent did something three steps in.

> ON THE EXAM
>
> An agent's failure is a trajectory, not a response. Without traces you cannot see which step went wrong, only that the answer was bad.

What to watch on a live agent

- **Steps per task.** A rising average means the agent is struggling before anyone complains.
- **Tool error rates**, per tool. One failing tool degrades every task that needs it.
- **Tasks hitting the step limit.** That is the runaway rate, and it should be near zero.
- **Escalation rate to humans**, which is the agent telling you what it cannot do.
- Module 30 covers monitoring agents properly. These are the signals worth having from day one.

> ON THE EXAM
>
> Steps per task is the leading indicator. It rises before quality falls, and before cost becomes visible on the bill.

Browser and Code Interpreter

- **Code Interpreter** runs code in a sandbox, which replaces Classic's AMAZON.CodeInterpreter action group.
- It lets an agent calculate, parse and transform reliably, rather than doing arithmetic in tokens.
- **Browser** lets an agent navigate and read web pages as a tool.
- Both expand what an agent can reach, so both are security decisions.
- Anything a browser can reach, a prompt-injected agent can be persuaded to reach.

> ON THE EXAM
>
> Code Interpreter fixes a real weakness: models are unreliable at arithmetic and completely reliable at writing code that does arithmetic.

Agents Classic to AgentCore, as AWS maps it

AGENTS CLASSIC	AGENTCORE
Action groups	Gateway. Needs explicit tool definition
AMAZON.CodeInterpreter action group	Code Interpreter
Session and memory configuration	Memory, short and long term, with strategies
Guardrails and policy on the agent	Bedrock Guardrails, plus policy enforcement on Gateway
Multi-agent collaboration and routing	Limited. See the next slide
Custom orchestrator	Supported via Runtime. Not available through harness

ON THE EXAM

AWS publishing this mapping is the clearest possible statement of direction. It is worth reading as a roadmap, not just a migration guide.

Multi-agent collaboration is limited today

- AWS states plainly that multi-agent collaboration on AgentCore is **limited**.
- The **supervisor pattern** is possible by exposing agents as **MCP tools**, so one agent calls another as a tool.
- **Routing-mode multi-agent is not straightforward today.**
- **Full multi-agent collaboration requires custom framework code.**
- Agents Classic had richer built-in collaboration roles and routing. That capability has not fully carried across.

> ON THE EXAM
>
> An unusually candid piece of AWS documentation. Worth trusting, and worth designing around rather than hoping.

MCP, and agent-as-tool

- **Model Context Protocol** is a standard way to expose tools to a model.
- An **agent exposed as an MCP tool** can be called by another agent, which is how supervision works today.
- It decouples tool providers from agent frameworks, so a tool serves any compatible agent.
- AgentCore Gateway is where those tool definitions are registered and governed.
- **AG-UI** was added in March 2026 for real-time bidirectional streaming, which suits voice agents.

> ON THE EXAM
>
> Agent-as-tool is the composable idea: if an agent is just a tool with judgement, agents compose the way tools do.

Guardrails around an agent

- A **Guardrail** filters what goes in and what comes out, as it does for any Bedrock call.
- With an agent it also matters **mid-loop**, because tool results become model input.
- **Prompt injection is worse here**, from module 16. Untrusted text can now trigger actions.
- A retrieved document or an API response can contain instructions the agent follows.
- Combine Guardrails with Policy and least-privilege tools. No single control is sufficient.

> ON THE EXAM
>
> An agent reads tool output as input. Anything that can write to a system the agent reads can attempt to steer it.

Sessions and separation

- AgentCore Runtime gives **per-session isolation**, so one user's session does not touch another's.
- **Long-term memory crosses sessions by design**, which is what makes scoping it correctly important.
- Memory must be scoped to a user or tenant, never shared by accident.
- Combine with identity delegation, so what the agent can reach matches who it is acting for.
- Module 26's multi-tenancy reasoning applies here, to memory as well as to documents.

> ON THE EXAM
>
> Long-term memory is a cross-session store of user information. Treat it with the same care as any other personal data store.

Frameworks and the Strands SDK

- **Strands Agents SDK** is an open-source agent framework from AWS, used with AgentCore Runtime.
- AgentCore is **framework-agnostic**, so other frameworks run on it too.
- That matters: you can adopt the runtime, memory or gateway without adopting a whole framework.
- A custom orchestrator deploys directly to Runtime, if you want full control of the loop.
- The platform provides the operational concerns. The framework provides the loop.

> ON THE EXAM
>
> The division is worth holding: your framework decides how the agent thinks, and AgentCore decides how it runs, remembers and is observed.

Limits, and why they matter

- An agent that cannot finish will keep trying. **Set a maximum number of steps.**
- Each step is a model call, so an unbounded loop consumes quota and budget until stopped.
- Set a **time limit** as well, because a slow tool can stall a loop without erroring.
- Decide what happens on the limit: give up, escalate to a human, or return partial work.
- Module 07's human review is the natural escalation path when an agent cannot proceed.

> ON THE EXAM
>
> The runaway agent is the characteristic failure of this architecture, and it is a cost failure before it is a quality one.

Human approval in an agent loop

- Some actions should **pause for a human**: refunds, deletions, anything irreversible.
- The agent proposes, a person approves, and only then does the action execute.
- That turns an unbounded risk into a reviewable one, without removing the agent's usefulness.
- Module 07's human review loop is the same pattern, applied to actions rather than predictions.
- Approve too much and people rubber-stamp. Reserve it for what genuinely warrants it.

ON THE EXAM

The rule worth applying: if you would not let a new joiner do it unsupervised on day one, do not let an agent do it unsupervised either.

Testing an agent

- Test the **tools independently** first. A broken tool looks exactly like a confused agent.
- Then test the agent against tasks with **known correct outcomes**, and read the traces.
- **Test the failure paths deliberately:** a tool that errors, a question outside scope, an impossible request.
- Confirm the step limit fires, and that what happens next is what you intended.
- Testing only the happy path tells you almost nothing about an agent.

> ON THE EXAM
>
> An agent's interesting behaviour is at the edges, because the middle is where it behaves like ordinary tool use.

Versioning an agent

- An agent is a **configuration**: instructions, tools, memory settings, model, and limits.
- **Any of those changes behaviour**, and most can change without a code deployment.
- So version the whole configuration, and test a change before it reaches users.
- Classic used **versions and aliases**. An alias points at a version, so promotion is an alias change.
- Bedrock has no deployment guardrails, from module 25. Your rollout is your own.

> ON THE EXAM
>
> The same problem as module 16's prompts: an agent's instructions are production behaviour that can change with no deployment.

Deploying an agent

- A framework-built agent deploys to **Runtime** as code or a container image.
- Gateway targets, Memory resources and Identity providers are **separate resources** it references.
- So deploying a new agent version does not necessarily change its tools or its memory.
- That separation is useful: you can change tool permissions without redeploying the agent.
- It also means a change to a shared Gateway affects every agent using it.

> ON THE EXAM
>
> The modularity cuts both ways. Independent deployment is a benefit; shared resources are a blast radius.

When you do not need an agent

- **A fixed sequence of steps is a workflow**, not an agent. Use Step Functions or a pipeline.
- **One tool call is tool use**, from module 16. It does not need a loop.
- **Answering from documents is retrieval**, from module 26. No actions are required.
- An agent earns its cost when **the sequence of steps is not known in advance**.
- If you can draw the flowchart, build the flowchart. It will be cheaper and far easier to debug.

> ON THE EXAM
>
> Agents are the most expensive and least predictable option in this domain. The bar for using one should be genuinely high.

Agent or Step Functions?

STEP FUNCTIONS, WHEN

The steps are known

- You can draw the flowchart.
- Deterministic, and cheap to run.
- Every path is visible and testable.
- Failures are ordinary engineering failures.

AGENT, WHEN

The steps are not known

- The path depends on what is found.
- The input space is open-ended.
- A fixed flowchart would have hundreds of branches.
- You accept variable cost and behaviour.

THE TIEBREAKER

A state machine with a model call at one step is often the better design, and it is the option teams skip entirely.

Amazon Q and packaged agents

- AWS also offers packaged agentic products for common jobs, rather than a platform to build on.
- Those suit **employee knowledge work**, where the requirement is permission-aware search and assistance.
- AgentCore suits **customer-facing and product-embedded** agents you are building yourself.
- Building a general assistant that already exists as a product is module 12's ladder, ignored.
- Check what is available before building. This area changes quickly.

> ON THE EXAM
>
> Module 12's question again: how little can you build? Sometimes the agent you need is a product someone already ships.

What an agent costs

COMPONENT	COST SHAPE
Model calls	The largest line for most agents. One per loop step
Growing context	History is resent each step, so later steps cost more. Module 16
Runtime	Consumption-based. Runtime instances bill EC2 in your account
Memory	Storage and retrieval of short and long term memory
Gateway and tools	Per invocation, plus whatever the downstream systems charge
Observability	CloudWatch ingestion and retention for traces and spans

ON THE EXAM

Model tokens dominate. Everything else is real and smaller, which is why step count is the number to control.

Symptoms, and what they usually mean

SYMPTOM	USUAL CAUSE
Agent loops without finishing	No step limit, or no clear completion condition
Cost far above expectation	Step count, and history resent each step
Agent calls the wrong tool	Tool descriptions are unclear. They are prompts, and they are read
Cannot tell why it did something	Observability not set up. Transaction Search and ADOT
Agent reaches data a user should not see	One service role, and no identity delegation
Multi-agent routing does not work as expected	Documented gap. Requires custom framework code
Agent forgets between conversations	Short-term memory only. Long-term memory not configured

ON THE EXAM

Row three is underrated: a tool's description is part of the prompt, and a vague one produces a confidently wrong tool choice.

An agent decides **what to do next**. Everything hard about them follows from that.

A team is starting a new project that needs an agent to triage IT support tickets, call internal APIs, and escalate to a human when it cannot resolve something.

They are choosing a platform. One engineer proposes Amazon Bedrock Agents, having used it eighteen months ago.

How should you advise them?

A Use Agents Classic. It is the documented Bedrock agent service

B Use AgentCore. Agents Classic is in maintenance mode, with a documented migration path

C Build on a SageMaker AI endpoint, since agents are not a managed capability

D Use a Bedrock knowledge base, which covers ticket triage

◆ PAUSE AND DECIDE

How should you advise them?

A. Use Agents Classic. It is the documented Bedrock agent service

It was, eighteen months ago. AWS now publishes a maintenance-mode page for it and a migration mapping to AgentCore. Starting new work there means migrating later.

B. Use AgentCore. Agents Classic is in maintenance mode, with a documented migration path

New project, no existing investment, and AWS has published both the maintenance status and the component-by-component successor. The choice is clear.

C. Build on a SageMaker AI endpoint, since agents are not a managed capability

Factually wrong. AgentCore is a managed agent platform, and an endpoint provides none of runtime, memory, gateway, identity or observability.

D. Use a Bedrock knowledge base, which covers ticket triage

Retrieval answers from documents. This needs to call internal APIs and escalate, which are actions, not passages.

THE SIGNAL

For the exam, know Agents Classic because the guide names it. For a new project, the maintenance page is the answer.

> A customer service agent handles a conversation well, but customers returning a week later find it remembers nothing about their previous issue, their preferences, or what was already tried.
>
> Within a single conversation, context is maintained correctly.

What is missing?

A. A longer session idle timeout on the runtime

B. A knowledge base containing previous conversation transcripts

C. A larger context window on the model

D. Long-term memory, persisting across sessions

◆ PAUSE AND DECIDE

What is missing?

A A longer session idle timeout on the runtime

A timeout extends one session. A week later is a new session by any configuration, and holding sessions open for weeks is not the mechanism.

B A knowledge base containing previous conversation transcripts

Would retrieve transcripts as documents, needing an ingestion cycle and returning passages rather than structured recall. Memory is the purpose-built path.

C A larger context window on the model

Context holds what is sent in this request. Nothing from last week is being sent, so a larger window has nothing extra to hold.

D Long-term memory, persisting across sessions

Short-term memory works, which is why single conversations are fine. Long-term memory is the component that persists across sessions.

THE SIGNAL

The scenario separates the two memory types precisely: within a conversation it works, across conversations it does not.

An internal agent can query the HR system on behalf of employees. It is configured with a single service role that has read access to all employee records.

During testing, an employee asks the agent for a colleague's salary. The agent retrieves and reports it.

What should be changed?

A Use identity delegation, so the agent acts with the requesting user's permissions

B Add a guardrail that blocks the word 'salary' in questions

C Add an instruction to the agent telling it not to disclose salaries

D Move the HR data into a knowledge base with metadata filtering

◆ PAUSE AND DECIDE

What should be changed?

A Use identity delegation, so the agent acts with the requesting user's permissions

The HR system already knows who may see what. Delegation makes the agent inherit the user's permissions, so the record is never returned to it.

B Add a guardrail that blocks the word 'salary' in questions

Blocks one phrasing of one question. The agent still holds access to every record, and 'what does she earn' is not the word salary.

C Add an instruction to the agent telling it not to disclose salaries

An instruction is a prompt, and prompts can be talked around. The underlying access is unchanged.

D Move the HR data into a knowledge base with metadata filtering

Replaces a live authoritative system with a stale index, and still requires per-user filtering to be correct. It solves the wrong problem.

THE SIGNAL

Options B and C try to make a model behave. Option A removes the capability. Only one of those survives a determined user.

> An architect designs a system with a supervisor agent routing work to five specialist agents, each with its own tools, using built-in routing on AgentCore. The design document assumes this is configuration.
>
> The team has no capacity to write and maintain custom framework code.

What should you tell them?

A Agree. Supervisor routing is a standard AgentCore configuration

B Agree, but use Agents Classic instead for the built-in collaboration roles

C Flag it. Routing-mode multi-agent is not straightforward today, and full collaboration needs custom code

D Flag it. AgentCore does not support more than one agent per account

◆ PAUSE AND DECIDE

What should you tell them?

A Agree. Supervisor routing is a standard AgentCore configuration

AWS documents the opposite. The supervisor pattern is possible via agent-as-tool over MCP; routing-mode multi-agent is explicitly called not straightforward.

B Agree, but use Agents Classic instead for the built-in collaboration roles

Classic did have richer collaboration, and it is in maintenance mode. Building new work there trades one problem for a worse one.

C Flag it. Routing-mode multi-agent is not straightforward today, and full collaboration needs custom code

AWS states this plainly in the migration guidance. With no capacity for custom framework code, the design needs revisiting before it is built.

D Flag it. AgentCore does not support more than one agent per account

Invented. Multiple agents are supported, and can even share a runtime with a shared session directory.

THE SIGNAL

B is the interesting trap: Classic genuinely is better at this, and choosing a maintenance-mode service for a new build is still wrong.

What to take from this module

- **Bedrock Agents is now Agents Classic, in maintenance.** Examinable, and superseded by AgentCore.
- **Action groups become Gateway. Session memory becomes Memory.** The mapping is published.
- **Multi-agent routing is limited today**, and full collaboration needs custom framework code.
- **Identity delegation** is how an agent stops being able to fetch anything for anyone.
- **Set step and time limits.** The runaway loop is this architecture's characteristic failure.

> ON THE EXAM
>
> Module 28 covers orchestration and CI/CD: pipelines, automation and the retraining loop.